

PROPOSAL PERANCANGAN SISTEM UJIAN TENGAH SEMESTER

Praktikum Object Oriented Programming

SmartPelayanan (SPLM)

"Satu Pintu Digital untuk Pengaduan & Pelayanan Masyarakat"

Sistem Informasi Pengaduan dan Layanan Administrasi
Masyarakat Berbasis Spring Boot

Bidang Pengembangan: Sistem Peningkatan Pelayanan Masyarakat

Nama Anggota	NIM
[Nama Anggota 1]	[NIM 1]
[Nama Anggota 2]	[NIM 2]
[Nama Anggota 3]	[NIM 3]
[Nama Anggota 4]	[NIM 4]

Kelompok : [Nomor Kelompok]
Dosen Pengampu : [Nama Dosen Pengampu]
Asisten Laboratorium : [Nama Asisten Laboratorium]

**PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INDUSTRI
INSTITUT TEKNOLOGI NASIONAL (ITENAS) BANDUNG
2026**

KATA PENGANTAR

Puji dan syukur kami panjatkan ke hadirat Tuhan Yang Maha Esa atas segala rahmat dan karunia-Nya sehingga proposal perancangan sistem ini dapat diselesaikan dengan baik. Proposal ini disusun dalam rangka memenuhi tugas Ujian Tengah Semester (UTS) Praktikum Object Oriented Programming.

Proposal ini mendeskripsikan rancangan sistem informasi bernama SmartPelayanan (SPLM), yaitu sebuah platform backend berbasis Spring Boot yang dirancang untuk memdigitalkan proses pengaduan dan layanan administrasi masyarakat. Di dalamnya tercakup latar belakang masalah, landasan teori OOP, desain database (ERD), rancangan class diagram, serta spesifikasi REST API secara menyeluruh.

Kami menyadari bahwa proposal ini masih memiliki kekurangan. Oleh karena itu, kami terbuka terhadap saran dan masukan yang membangun dari dosen pengampu dan asisten laboratorium demi kesempurnaan laporan dan implementasi sistem ini.

Bandung, 2026

Tim Pengembang Kelompok [Nomor Kelompok]

DAFTAR ISI

Kata Pengantar	2
Daftar Isi	3
BAB I – Pendahuluan	4
1.1 Latar Belakang	4
1.2 Rumusan Masalah	4
1.3 Tujuan Pengembangan	5
1.4 Batasan Sistem	5
1.5 Manfaat Sistem.....	5
BAB II – Landasan Teori	6
2.1 Konsep Object Oriented Programming (OOP)	6
2.2 Spring Boot dan Arsitektur Berlapis	7
2.3 REST API dan HTTP	7
2.4 Basis Data Relasional dan JPA	8
BAB III – Perancangan Sistem	9
3.1 Gambaran Umum Sistem	9
3.2 Desain ERD (Entity Relationship Diagram)	9
3.3 Rancangan Class Diagram	11
3.4 Rancangan REST API.....	13
3.5 Alur Kerja Sistem (Workflow)	16
BAB IV – Penutup.....	17
4.1 Kesimpulan	17
4.2 Rencana Implementasi	17

BAB I

PENDAHULUAN

1.1 Latar Belakang

Pelayanan publik merupakan salah satu tanggung jawab utama pemerintah kepada masyarakat. Namun, hingga saat ini banyak instansi pemerintah di tingkat kelurahan dan kecamatan yang masih mengandalkan proses manual dalam mengelola pengaduan warga maupun permohonan layanan administrasi seperti pembuatan surat keterangan, surat domisili, dan izin usaha mikro.

Proses manual tersebut menimbulkan sejumlah permasalahan, di antaranya: antrean panjang di kantor pelayanan, risiko kehilangan berkas, ketidakjelasan status proses, serta rendahnya transparansi bagi warga selaku pemohon. Kondisi ini berdampak pada menurunnya kepercayaan masyarakat terhadap kualitas pelayanan pemerintah setempat.

Perkembangan teknologi informasi, khususnya platform berbasis web dan REST API, membuka peluang untuk menghadirkan solusi digital yang mampu memusatkan, memvirtualkan, dan meningkatkan efisiensi proses pelayanan tersebut. Dengan memanfaatkan paradigma Object Oriented Programming (OOP) melalui framework Spring Boot, dimungkinkan untuk membangun sistem yang terstruktur, skalabel, dan mudah dipelihara.

Berdasarkan latar belakang di atas, kami mengusulkan pengembangan sistem bernama "SmartPelayanan (SPLM)" — sebuah platform backend satu pintu digital yang memungkinkan warga mengajukan pengaduan dan permohonan layanan administrasi secara online, sementara petugas/admin dapat memantau, memproses, dan memperbarui status secara real-time melalui antarmuka REST API.

1.2 Rumusan Masalah

Berdasarkan uraian latar belakang di atas, rumusan masalah yang dihadapi adalah sebagai berikut:

1. Bagaimana membangun platform backend (REST API) yang dapat menjadi satu pintu terpadu bagi warga untuk menyampaikan pengaduan dan mengajukan permohonan layanan administrasi secara digital?
2. Bagaimana menyediakan mekanisme pelacakan status (tracking) permohonan secara real-time sehingga warga dapat mengetahui perkembangan pengajuannya tanpa harus mendatangi kantor?

3. Bagaimana mengelola dan mengarsipkan data pengaduan beserta dokumen pendukung secara sistematis agar admin dapat memonitor dan merespons dengan cepat?
4. Bagaimana menerapkan prinsip-prinsip OOP (Encapsulation, Inheritance, Polymorphism, Abstraction) secara nyata dalam arsitektur sistem SmartPelayanan?

1.3 Tujuan Pengembangan

Tujuan dari pengembangan sistem SmartPelayanan adalah:

1. Membangun aplikasi backend REST API berbasis Spring Boot yang terstruktur menggunakan pola Layered Architecture (Controller – Service – Repository).
2. Mengimplementasikan manajemen pengaduan warga yang mencakup operasi CRUD lengkap beserta pembaruan status proses.
3. Mengimplementasikan modul layanan administrasi yang mendukung pengajuan dan pengelolaan dokumen permohonan warga.
4. Menerapkan konsep OOP secara terstruktur dan terdokumentasi sesuai standar penilaian UTS Praktikum OOP.
5. Menyediakan dokumentasi perancangan sistem yang komprehensif meliputi ERD, Class Diagram, dan spesifikasi API.

1.4 Batasan Sistem

Agar pengembangan tetap terfokus, sistem SmartPelayanan dibatasi pada hal-hal berikut:

- Lingkup pengembangan mencakup layer Backend (REST API) saja; antarmuka pengguna (frontend) tidak termasuk dalam cakupan proyek ini.
- Sistem tidak mengintegrasikan modul pembayaran (payment gateway) apapun.
- Autentikasi dan otorisasi diimplementasikan menggunakan mekanisme Role-Based Access Control (RBAC) sederhana dengan role: WARGA dan ADMIN.
- Penyimpanan dokumen pendukung menggunakan referensi path/URL file; belum mencakup penyimpanan cloud (object storage).
- Sistem diuji secara lokal menggunakan Postman/Insomnia; belum mencakup pengujian beban (load testing).

1.5 Manfaat Sistem

Manfaat yang diharapkan dari implementasi SmartPelayanan antara lain:

- Bagi Warga: Kemudahan mengajukan pengaduan atau permohonan layanan kapan saja dan di mana saja tanpa harus antri di kantor.
- Bagi Admin/Petugas: Pengelolaan pengaduan dan layanan lebih terorganisir, data terpusat, dan proses monitoring lebih efisien.
- Bagi Akademik: Sebagai media penerapan nyata konsep OOP, Spring Boot, dan desain sistem dalam konteks pelayanan publik.

BAB II

LANDASAN TEORI

2.1 Konsep Object Oriented Programming (OOP)

Object Oriented Programming (OOP) adalah paradigma pemrograman yang mengorganisasikan kode ke dalam unit-unit yang disebut class dan object. Paradigma ini berfokus pada pemodelan entitas dunia nyata ke dalam bentuk objek yang memiliki atribut (data) dan perilaku (method). Empat pilar utama OOP yang diterapkan dalam SmartPelayanan adalah:

a. Encapsulation (Enkapsulasi)

Encapsulation adalah mekanisme penyembunyian detail implementasi internal sebuah class dari dunia luar. Dalam SmartPelayanan, seluruh field pada class Entity seperti User dan Pengaduan dideklarasikan sebagai private, dan akses dari luar dilakukan melalui method getter dan setter. Hal ini mencegah manipulasi data secara tidak terkontrol dan menjaga integritas data objek.

b. Inheritance (Pewarisan)

Inheritance memungkinkan sebuah class (subclass) mewarisi atribut dan method dari class lain (superclass). Dalam sistem ini, dirancang sebuah abstract class bernama BaseEntity yang memiliki field audit umum berupa id (UUID), createdAt, dan updatedAt. Class-class entitas utama seperti User, Pengaduan, dan LayananAdministrasi meng-extend BaseEntity sehingga otomatis memiliki field audit tersebut tanpa perlu mendefinisikan ulang.

c. Polymorphism (Polimorfisme)

Polymorphism memungkinkan satu method berperilaku berbeda berdasarkan konteks objek yang memanggilnya. Dalam SmartPelayanan, polimorfisme diterapkan melalui mekanisme interface: setiap modul bisnis mendefinisikan kontrak atau interface (misal: PengaduanService), dan implementasi nyatanya ditempatkan di kelas terpisah (misal: PengaduanServiceImpl). Spring Boot secara dinamis menginjeksikan implementasi tersebut (Dependency Injection), sehingga perubahan implementasi tidak mempengaruhi layer Controller.

d. Abstraction (Abstraksi)

Abstraction adalah proses menyembunyikan detail kompleks dan hanya mengekspos fungsionalitas esensial kepada pengguna. Dalam sistem ini, abstraksi diwujudkan melalui penggunaan interface sebagai service contract. Layer Controller hanya berinteraksi dengan interface service tanpa mengetahui detail implementasinya.

Selain itu, penggunaan abstract class BaseEntity juga merupakan contoh abstraksi untuk menghindari duplikasi kode audit.

2.2 Spring Boot dan Arsitektur Berlapis

Spring Boot adalah framework berbasis Java yang menyederhanakan pengembangan aplikasi berbasis Spring Framework dengan konfigurasi otomatis (auto-configuration) dan prinsip Convention over Configuration. Spring Boot memungkinkan pengembang untuk langsung fokus pada logika bisnis tanpa harus mengonfigurasi server atau dependency secara manual.

SmartPelayanan mengadopsi Layered Architecture (Arsitektur Berlapis) tiga lapis:

- Controller Layer (@RestController): Menerima HTTP request, memvalidasi input, mendelegasikan proses ke Service Layer, lalu mengemas response dalam format JSON.
- Service Layer (@Service + Interface): Memuat seluruh logika bisnis aplikasi. Setiap service direpresentasikan sebagai interface yang diimplementasikan oleh kelas ServiceImpl.
- Repository Layer (@Repository): Berinteraksi langsung dengan database menggunakan Spring Data JPA. Meng-extend JpaRepository<Entity, ID> untuk mendapatkan operasi CRUD built-in.

2.3 REST API dan HTTP

REST (Representational State Transfer) adalah gaya arsitektur komunikasi antara client dan server melalui protokol HTTP. API RESTful menggunakan HTTP method yang bermakna dan mengidentifikasi sumber daya (resource) melalui URL. SmartPelayanan mengimplementasikan HTTP method berikut:

- GET — Mengambil/membaca data resource.
- POST — Membuat resource baru.
- PUT — Memperbarui data resource secara menyeluruh.
- PATCH — Memperbarui sebagian field resource (misal: status saja).
- DELETE — Menghapus resource.

Seluruh response dari API SmartPelayanan menggunakan format JSON (JavaScript Object Notation) dengan struktur baku yang terdiri dari field: status, message, dan data.

2.4 Basis Data Relasional dan JPA

SmartPelayanan menggunakan basis data relasional (MySQL atau PostgreSQL) untuk menyimpan data secara persisten. Relasi antar tabel

direpresentasikan menggunakan Foreign Key sesuai dengan kaidah normalisasi database.

Akses database dilakukan melalui Java Persistence API (JPA) dengan implementasi Hibernate sebagai ORM (Object Relational Mapping). Dengan JPA, class Java (`@Entity`) secara otomatis ter-mapping dengan tabel database, sehingga developer dapat berinteraksi dengan database menggunakan object Java tanpa menulis query SQL secara manual untuk operasi-operasi dasar.

BAB III PERANCANGAN SISTEM

3.1 Gambaran Umum Sistem

SmartPelayanan adalah sistem informasi backend yang berfungsi sebagai jembatan digital antara warga dan petugas administrasi. Sistem ini terdiri dari dua aktor utama: WARGA (pengguna yang mengajukan pengaduan/layanan) dan ADMIN (petugas yang memproses dan merespons setiap permohonan). Berikut adalah gambaran alur penggunaan sistem secara umum:

- Warga melakukan registrasi dan login untuk mendapatkan akses ke sistem.
- Warga dapat mengajukan pengaduan baru atau permohonan layanan administrasi.
- Warga dapat memantau status pengaduan/permohonannya secara real-time.
- Admin menerima notifikasi pengajuan baru dan dapat memperbarui status pengaduan.
- Admin dapat mengelola data kategori layanan dan merekap seluruh data pengaduan.

3.2 Desain ERD (Entity Relationship Diagram)

Perancangan database SmartPelayanan terdiri dari enam tabel utama yang saling berelasi. Berikut adalah deskripsi lengkap setiap entitas beserta atributnya:

Nama Tabel	Primary Key	Foreign Key	Atribut / Field
tb_user	id (UUID)	-	nama_lengkap (VARCHAR), email (VARCHAR, UNIQUE), password (VARCHAR, BCrypt), no_telp (VARCHAR), alamat (TEXT), role (ENUM: WARGA/ADMIN), status_aktif (BOOLEAN), created_at (TIMESTAMP), updated_at (TIMESTAMP)
tb_kategori_layanan	id (INT, AUTO INCREMENT)	-	nama_kategori (VARCHAR), deskripsi (TEXT), is_active (BOOLEAN), created_at (TIMESTAMP)
tb_pengaduan	id (UUID)	user_id → tb_user.id	judul (VARCHAR), deskripsi (TEXT), lokasi (VARCHAR), status (ENUM: BARU/DIPROSES/SELESAI/DITOLAK), prioritas (ENUM: RENDAH/SEDANG/TINGGI), foto_bukti (VARCHAR/URL), tanggal_kejadian (DATE), catatan_admin (TEXT), created_at (TIMESTAMP), updated_at (TIMESTAMP)
tb_layanan_administrasi	id (UUID)	user_id → tb_user.id kategori_id → tb_kategori_layanan.id	nomor_permohonan (VARCHAR, UNIQUE), keperluan (TEXT), dokumen_pendukung (VARCHAR/URL), status (ENUM: MENUNGGU/DIPROSES/SELESAI/DITOLAK), catatan_petugas (TEXT),

			tgl diajukan (DATE), tgl selesai (DATE), created_at (TIMESTAMP), updated_at (TIMESTAMP)
tb_respon_pengaduan	id (UUID)	pengaduan_id → tb_pengaduan.id admin_id → tb_user.id	isi_respon (TEXT), tgl_respon (TIMESTAMP)
tb_riwayat_status	id (UUID)	referensi_id (UUID, polymorphic) admin_id → tb_user.id	tipe_referensi (ENUM: PENGADUAN/LAYANAN), status_lama (VARCHAR), status_baru (VARCHAR), alasan_perubahan (TEXT), changed_at (TIMESTAMP)

Relasi Antar Entitas (ERD Relationships)

Berikut penjabaran relasi antar tabel dalam sistem SmartPelayanan:

- tb_user (1) —< tb_pengaduan (N) : Satu warga dapat mengajukan banyak pengaduan. Relasi One-to-Many melalui FK user_id.
- tb_user (1) —< tb_layanan_administrasi (N) : Satu warga dapat mengajukan banyak permohonan layanan. Relasi One-to-Many melalui FK user_id.
- tb_kategori_layanan (1) —< tb_layanan_administrasi (N) : Satu kategori dapat digunakan oleh banyak permohonan. Relasi One-to-Many melalui FK kategori_id.
- tb_pengaduan (1) —< tb_respon_pengaduan (N) : Satu pengaduan dapat memiliki banyak respon dari admin. Relasi One-to-Many.
- tb_user (1) —< tb_respon_pengaduan (N) : Satu admin dapat memberikan banyak respon. Relasi One-to-Many melalui FK admin_id.
- tb_riwayat_status : Mencatat semua perubahan status untuk pengaduan dan layanan (polymorphic reference).

3.3 Rancangan Class Diagram

Sistem SmartPelayanan dirancang menggunakan pola Layered Architecture dengan pemisahan tanggung jawab yang jelas. Berikut adalah deskripsi class-class yang direncanakan beserta keterkaitan antar kelasnya:

Nama Class	Tipe	Atribut	Method	Relasi OOP
BaseEntity	@MappedSuperclass (Abstract Class)	- id: UUID - createdAt: LocalDateTime - updatedAt: LocalDateTime	+ getId(): UUID + getCreatedAt(): LocalDateTime + getUpdatedAt(): LocalDateTime [Auto-managed oleh JPA @PrePersist/@PreUpdate]	Superclass dari: User, Pengaduan, LayananAdministrasi, ResponPengaduan, RiwayatStatus
User	@Entity	- namaLengkap:	+ getNamaLengkap():	One-to-Many →

	(Extends BaseEntity)	String - email: String - password: String - noTelp: String - alamat: String - role: RoleEnum - statusAktif: Boolean	String + setPassword(pwd): void + getRole(): RoleEnum + isStatusAktif(): Boolean	Pengaduan LayananAdministrasi ResponPengaduan
Pengaduan	@Entity (Extends BaseEntity)	- judul: String - deskripsi: String - lokasi: String - status: StatusEnum - prioritas: PrioritasEnum - fotoBukti: String - tanggalKejadian: LocalDate - catatanAdmin: String - user: User	+ getStatus(): StatusEnum + setStatus(s): void + getUser(): User + isOpen(): boolean	Many-to-One → User One-to-Many → ResponPengaduan RiwayatStatus
KategoriLayanan	@Entity	- id: Integer - namaKategori: String - deskripsi: String - isActive: Boolean	+ getNamaKategori(): String + isActive(): Boolean	One-to-Many → LayananAdministrasi
LayananAdministrasi	@Entity (Extends BaseEntity)	- nomorPermohonan: String - keperluan: String - dokumenPendukung: String - status: StatusEnum - catatanPetugas: String - tglDiajukan: LocalDate - tglSelesai: LocalDate - user: User - kategori: KategoriLayanan	+ getStatus(): StatusEnum + setStatus(s): void + generateNomorPermohonan(): String	Many-to-One → User Many-to-One → KategoriLayanan
PengaduanService (Interface)	Interface (Service Contract)	-	+ findAll(): List<PengaduanDTO> + findById(id): PengaduanDTO + save(req): PengaduanDTO + update(id, req): PengaduanDTO + updateStatus(id, status): void + delete(id): void	Implemented by: PengaduanServiceImpl
PengaduanServiceImpl	@Service (Implements PengaduanService)	- pengaduanRepository: PengaduanRepository - riwayatStatusRepository:	Override semua method dari PengaduanService + validateOwnership(id, userId): void	Implements: PengaduanService Uses: PengaduanRepository

		tory: RiwayatStatusRepository		
PengaduanController	@RestController @RequestMapping ("/api/v1/pengaduan")	- pengaduanService: PengaduanService	+ getAllPengaduan(): ResponseEntity + getPengaduanById(id): ResponseEntity + createPengaduan(req): ResponseEntity + updatePengaduan(id,req): ResponseEntity + deleteById(id): ResponseEntity + updateStatus(id,status): ResponseEntity	Depends on: PengaduanService (via Dependency Injection)
PengaduanRepository	@Repository (Extends JpaRepository)	-	Inherited CRUD + findById(userId) + findByStatus(status) + findByPrioritas(p)	Used by: PengaduanServiceImpl
ApiResponse<T>	Generic Wrapper Class (DTO)	- status: int - message: String - data: T - timestamp: LocalDateTime	+ success(data): ApiResponse<T> + error(msg): ApiResponse<T>	Digunakan oleh semua Controller sebagai wrapper response

Enum yang Digunakan

Berikut adalah enum (tipe data konstanta) yang digunakan dalam sistem:

- RoleEnum: WARGA, ADMIN — Mendefinisikan peran pengguna dalam sistem.
- StatusPengaduanEnum: BARU, DIPROSES, SELESAI, DITOLAK — Status progres pengaduan warga.
- StatusLayananEnum: MENUNGGU, DIPROSES, SELESAI, DITOLAK — Status progres permohonan layanan administrasi.
- PrioritasEnum: RENDAH, SEDANG, TINGGI — Tingkat urgensi dari sebuah pengaduan.

3.4 Rancangan REST API

Seluruh endpoint SmartPelayanan dikelompokkan dalam base URL: /api/v1. Format response menggunakan wrapper standar { "status": int, "message": String, "data": Object, "timestamp": String }. Autentikasi menggunakan Bearer Token (JWT) yang disertakan di header Authorization.

a. Auth (Autentikasi)

HTTP	Endpoint URL	Request Body /	Response	Deskripsi
------	--------------	----------------	----------	-----------

Method		Param		
POST	/api/v1/auth/register	Body: { namaLengkap, email, password, noTelp, alamat }	201 Created { token, user }	Registrasi akun warga baru. Password di-hash BCrypt sebelum disimpan.
POST	/api/v1/auth/login	Body: { email, password }	200 OK { token, refresh_token }	Login dan mendapatkan JWT access token untuk mengakses endpoint terproteksi.
POST	/api/v1/auth/refresh	Header: RefreshToken	200 OK { token }	Memperbarui access token yang sudah kadaluarsa menggunakan refresh token.
GET	/api/v1/auth/profile	Header: Bearer Token	200 OK { user detail }	Mengambil profil akun pengguna yang sedang login.

b. Pengaduan Warga

HTTP Method	Endpoint URL	Request Body / Param	Response	Deskripsi
GET	/api/v1/pengaduan	Header: Bearer Token Query: status, prioritas, page, size	200 OK { data: [pengaduan], pagination }	Mengambil semua data pengaduan. Admin mendapat semua data; Warga hanya mendapat miliknya sendiri.
GET	/api/v1/pengaduan/{id}	Path: id (UUID)	200 OK { detail pengaduan }	Mengambil detail satu pengaduan berdasarkan ID unik.
POST	/api/v1/pengaduan	Body: { judul, deskripsi, lokasi, prioritas, tanggalKejadian, fotoBukti? }	201 Created { pengaduan baru }	Warga mengajukan laporan pengaduan baru. Status otomatis diatur BARU.
PUT	/api/v1/pengaduan/{id}	Path: id Body: { judul, deskripsi, lokasi, prioritas }	200 OK { pengaduan updated }	Warga dapat mengedit isi pengaduannya selama status masih BARU.
PATCH	/api/v1/pengaduan/{id}/status	Path: id Body: { status, catatanAdmin }	200 OK { status updated }	Admin memperbarui status pengaduan dan mencatat keterangan. Perubahan status direkam di riwayat.
DELETE	/api/v1/pengaduan/{id}	Path: id (UUID)	200 OK { message }	Warga menghapus pengaduannya (hanya jika status masih BARU). Admin dapat menghapus kapan saja.
GET	/api/v1/pengaduan/{id}/respon	Path: id	200 OK { data: [respon] }	Mengambil semua respon/balasan admin untuk satu pengaduan tertentu.
POST	/api/v1/pengaduan/{id}/respon	Path: id Body: { isiRespon }	201 Created { respon baru }	Admin menambahkan respon/tanggapan tertulis terhadap pengaduan warga.

c. Layanan Administrasi

HTTP Method	Endpoint URL	Request Body / Param	Response	Deskripsi
GET	/api/v1/layanan	Header: Bearer Token Query: status, kategori, page, size	200 OK { data: [layanan], pagination }	Mengambil daftar semua permohonan layanan administrasi.
GET	/api/v1/layanan/{id}	Path: id (UUID)	200 OK { detail permohonan }	Mengambil detail satu permohonan layanan administrasi.
POST	/api/v1/layanan	Body: { kategoriId, keperluan, dokumenPendukung }	201 Created { nomor permohonan, detail }	Warga mengajukan permohonan layanan baru. Nomor permohonan auto-generated.
PATCH	/api/v1/layanan/{id}/status	Path: id Body: { status, catatanPetugas, tglSelesai? }	200 OK { status updated }	Admin memperbarui status permohonan layanan administrasi.
DELETE	/api/v1/layanan/{id}	Path: id	200 OK { message }	Warga membatalkan permohonannya (hanya jika status MENUNGGU).

d. Kategori Layanan (Admin Only)

HTTP Method	Endpoint URL	Request Body / Param	Response	Deskripsi
GET	/api/v1/kategori	-	200 OK { data: [kategori] }	Mengambil semua daftar kategori layanan administrasi yang tersedia.
POST	/api/v1/kategori	Body: { namaKategori, deskripsi }	201 Created { kategori baru }	Admin membuat kategori layanan baru (misal: Surat Domisili, Izin Usaha, dll).
PUT	/api/v1/kategori/{id}	Path: id Body: { namaKategori, deskripsi, isActive }	200 OK { kategori updated }	Admin mengubah informasi atau menonaktifkan kategori layanan.
DELETE	/api/v1/kategori/{id}	Path: id	200 OK { message }	Admin menghapus kategori layanan (hanya jika tidak ada layanan aktif yang terkait).

e. Laporan & Statistik (Admin Only)

HTTP Method	Endpoint URL	Request Body / Param	Response	Deskripsi
GET	/api/v1/dashboard/stats	Header: Bearer Token (Admin)	200 OK { total, per status, per kategori }	Mengambil ringkasan statistik seluruh sistem: total pengaduan, total layanan, breakdown per status, dll.

GET	/api/v1/riwayat-status/{id}	Path: id (UUID permohonan/pengaduan)	200 OK { data: [riwayat] }	Mengambil seluruh riwayat perubahan status dari satu pengaduan atau layanan untuk keperluan audit trail.
------------	-----------------------------	--------------------------------------	-------------------------------	--

3.5 Alur Kerja Sistem (Workflow)

Berikut adalah alur kerja utama penggunaan SmartPelayanan dari perspektif kedua aktor:

Alur Pengajuan Pengaduan oleh Warga:

1. Warga melakukan registrasi melalui POST /api/v1/auth/register.
2. Warga login melalui POST /api/v1/auth/login dan mendapatkan JWT token.
3. Warga mengajukan pengaduan melalui POST /api/v1/pengaduan dengan melampirkan detail masalah.
4. Sistem otomatis menetapkan status = BARU dan menyimpan data ke database.
5. Warga dapat memantau status pengaduan melalui GET /api/v1/pengaduan/{id}.

Alur Penanganan Pengaduan oleh Admin:

1. Admin login menggunakan akun admin dan mendapatkan JWT token.
2. Admin melihat seluruh daftar pengaduan melalui GET /api/v1/pengaduan.
3. Admin memproses pengaduan dan mengubah status menjadi DIPROSES melalui PATCH /api/v1/pengaduan/{id}/status.
4. Admin menambahkan respon/tanggapan melalui POST /api/v1/pengaduan/{id}/respon.
5. Setelah selesai ditangani, admin mengubah status menjadi SELESAI. Seluruh perubahan status tercatat di tb_riwayat_status.

BAB IV PENUTUP

4.1 Kesimpulan

SmartPelayanan (SPLM) adalah solusi backend digital yang dirancang untuk memodernisasi proses pengaduan dan pelayanan administrasi masyarakat. Melalui proposal ini, telah dijabarkan perancangan sistem yang komprehensif mencakup:

- Latar belakang dan justifikasi kebutuhan sistem berbasis masalah pelayanan manual yang ada.
- Landasan teori OOP (Encapsulation, Inheritance, Polymorphism, Abstraction) beserta penerapannya dalam Spring Boot.
- Desain ERD dengan 6 tabel yang saling berelasi secara terstruktur dan ternormalisasi.
- Rancangan Class Diagram dengan 10 class yang menerapkan pola Layered Architecture dan prinsip OOP secara konsisten.
- Spesifikasi REST API yang mencakup 23 endpoint terkelompok dalam 5 modul fungsional.

Dengan rancangan yang terstruktur ini, implementasi sistem diharapkan dapat berjalan lancar dan memenuhi seluruh kriteria penilaian Ujian Tengah Semester Praktikum OOP.

4.2 Rencana Implementasi

Berikut adalah rencana tahapan implementasi sistem SmartPelayanan:

Tahapan	Target Waktu	Aktivitas
Tahap 1	Minggu 1	Setup proyek Spring Boot, konfigurasi database, implementasi BaseEntity, User entity, dan AuthController (Login & Register).
Tahap 2	Minggu 2	Implementasi modul Pengaduan: Entity, Repository, Service Interface, ServiceImpl, dan Controller dengan operasi CRUD lengkap.
Tahap 3	Minggu 3	Implementasi modul Layanan Administrasi, KategoriLayanan, dan ResponPengaduan beserta seluruh endpointnya.
Tahap 4	Minggu 4	Implementasi RiwayatStatus, Dashboard Stats, pengujian end-to-end menggunakan Postman, dokumentasi API

		(Swagger), dan refactoring kode.
--	--	----------------------------------

Demikian proposal perancangan sistem SmartPelayanan (SPLM) ini disusun. Kami berkomitmen untuk mengimplementasikan sistem ini sesuai rancangan yang telah diajukan dengan mengedepankan kualitas kode, penerapan OOP yang benar, dan dokumentasi yang memadai. Apabila terdapat masukan atau revisi dari dosen pengampu dan asisten laboratorium, kami siap melakukan penyesuaian.

Bandung, 2026

Tim SmartPelayanan — Kelompok [Nomor Kelompok]